

Matricola: _____

Nome: _____

Cognome: _____

ESAME Programmazione Avanzata e Parallela

13 luglio 2026

L'esame consiste in due parti:

- La **prima parte** consiste in 10 domande a risposta multipla sugli argomenti del corso. Ogni domanda può ricevere un punteggio massimo di *due* punti. Affinché una risposta sia considerata valida la scelta *deve essere motivata* in modo breve (non saranno infatti parte della consegna eventuali fogli "di brutta" o aggiuntivi). Una risposta errata o non motivata riceverà *zero* punti. *Il punteggio minimo è 12 punti su 20.*
- La **seconda parte** consiste in due esercizi di programmazione. Ogni esercizio può avere un punteggio massimo di *cinque* punti. Potete commentare brevemente il codice per chiarire le vostre scelte implementative. *Il punteggio minimo è 6 punti su 10.*

IMPORTANTE: Per raggiungere la sufficienza è necessario raggiungere il punteggio minimo in entrambe le parti.

Parte 1

Domanda 1

Si supponga di avere il seguente Makefile:

```
CC = gcc
```

```
CFLAGS = -Wall -O3
```

```
main: a.o b.o c.o
```

```
$(CC) $(CFLAGS) MISSING1 -o $@
```

```
% .o: %.c
```

```
$(CC) $(CFLAGS) -c MISSING2
```

Per rendere corretto il makefile cosa è necessario sostituire a MISSING1 e MISSING2? Serve fare in modo che il target `main` crei un eseguibile con tutti i file oggetto elencati nei prerequisiti e che il target per creare il file oggetto li crei con lo stesso nome del file sorgente corrispondente.

☐ \$< sia per MISSING1 che per MISSING2

☐ \$@ per MISSING1 e \$^ per MISSING2

☐ \$^ per MISSING1 e \$< per MISSING2

☐ \$< per MISSING1 e \$@ per MISSING2

Domanda 2

Si supponga di avere questo frammento di codice C:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        if (a[i] < a[i+1]) {  
            M[i*n + j] = 0;  
        }  
    }  
}
```

e si assuma di avere variabili M , i , j , a e n definite e del tipo corretto. La variabile a punta a un vettore di `int` di $n + 1$ elementi generati casualmente tra il minimo e il massimo valore rappresentabile. La variabile M punta a una matrice $n \times n$ di `float` rappresentata in forma row-major.

Quale delle seguenti è la versione branchless del codice precedente? La versione branchless migliorerebbe le prestazioni per valori di n elevati? Si assuma che il compilatore non ottimizzi.

`int c = (a[i] < a[i+1]);`

☐ `M[i*n+j] = c*M[i*n+j];`

Migliorerebbe le prestazioni

`int c = (a[i] >= a[i+1]);`

☐ `if(!c) M[i*n+j] = (1-c)*M[i*n+j];`

Migliorerebbe le prestazioni

`int c = (a[i] < a[i+1]);`

☐ `M[i*n+j] = c*M[i*n+j];`

Non migliorerebbe le prestazioni

`int c = (a[i] >= a[i+1]);`

☐ `if(!c) M[i*n+j] = (1-c)*M[i*n+j];`

Non migliorerebbe le prestazioni

Domanda 3

Si consideri una lista concatenata in cui i nodi sono rappresentati dalla seguente struttura:

```
struct node {  
    int key;  
    struct node * next;  
};
```

E si supponga che una operazione comune sia di accedere a tutti i valori `key` in sequenza. Si supponga di poter effettuare una operazione di “compattazione” dei nodi in modo da assicurare che nodi successivi nella lista siano anche consecutivi in memoria, ovvero se un nodo è all’indirizzo `addr` il nodo successivo sarà all’indirizzo `addr + sizeof(struct node)`. Quali vantaggi avrebbe questa operazione di “compattazione”? Rispetto a un vettore di `int` contenente solo i valori `key` quali sarebbero gli svantaggi?

- | | |
|---|--|
| ☐ Il nodo successivo sarà spesso nella stessa linea di cache. Vi è comunque overhead di un puntatore che occupa spazio nella linea di cache e che è invece assente nel vettore | ☐ Il nodo successivo sarà sempre nella stessa linea di cache. Non vi sono svantaggi rispetto all’uso di un vettore |
| ☐ Permette di rimuovere padding interno ai nodi, come svantaggio rimane il padding tra i nodi che è assente nel vettore | ☐ Data la contiguità dei nodi sarà possibile salvare un solo puntatore per ogni linea di cache. Diversamente dal vettore, dobbiamo esplicitamente rimuovere il puntatore dalla linea di cache |
-
-
-

Domanda 4

Quale delle seguenti affermazioni sulle estensioni SIMD presenti nelle comuni architetture (e.g., AVX2 e AVX512 su x86_64 e NEON e SVE su ARM64) è corretta?

- | | |
|--|---|
| ☐ Due estensioni che fanno uso di registri di uguale dimensione supporteranno esattamente le stesse operazioni | ☐ L'aumento del numero di bit nei registri vettoriale (e.g., da 256 a 512) permette di aumentare il numero di core che l'architettura può sfruttare |
| ☐ L'utilizzo di registri di dimensione maggiore permette di utilizzare tipi di dato floating point con precisione aumentata interpretando l'intero registro come un unico numero floating point | ☐ In aggiunta ad aumentare le dimensioni dei registri, revisioni successive delle estensioni possono aggiungere operazioni effettuabili sui registri |
-
-
-

Domanda 5

Quale delle seguenti affermazioni sull'I/O è corretta?

- | | |
|--|--|
| ☐ <code>write</code> e <code>read</code> operano su buffer che devono essere di dimensione un multiplo esatto della dimensione della pagina del sistema | ☐ <code>ftell</code> permette di ottenere la posizione all'interno del file del primo carattere di controllo non stampabile |
| ☐ <code>read</code> e <code>write</code> ritornano il numero di byte rispettivamente letti o scritti | ☐ <code>mmap</code> richiede come uno degli argomenti il numero di pagine (non di byte) da mappare, dato che le allocazioni con <code>mmap</code> sono sempre per pagine e non per byte |
-
-
-

Domanda 6

Dato il seguente codice C facente uso di OpenMP:

```
float f(float * M, int * q, int n)
{
    float * v;
    float res = 1;
    #pragma omp parallel
    {
        #pragma TODO1
        {
            v = (float *) malloc(sizeof(float)*n);
        }
        #pragma omp for
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (M[i*n + j] > q[i]) {
                    v[i] = M[i*n + j];
                }
            }
        }
        #pragma TODO2
        for (int i = 0; i < n; i++) {
            res *= v[i];
        }
        #pragma TODO3
        {
            free(v);
        }
    }
    return res;
}
```

Si supponga che M sia una matrice $n \times n$ e q un vettore di n elementi. Al posto di TODO1, TODO2 e TODO3 che codice può essere inserito affinché il programma funzioni correttamente?

- | | |
|--|--|
| ☐ omp critical per TODO1, | ☐ omp single per TODO1 e TODO3 e |
| ☐ omp for reduction (*:res) per TODO2 | ☐ omp for reduction (*:res) per TODO2 |
| ☐ e omp single per TODO3 | |
| ☐ omp single per TODO1 e TODO3 e | ☐ omp critical per TODO1 e TODO3 e |
| ☐ omp for critical per TODO2 | ☐ omp for reduction (*:res) per TODO2 |
-
-
-

Domanda 7

Dato il seguente codice Python:

```
class A:

    def __init__(self, f):
        self.f = f

    def __add__(self, a):
        g = lambda x: self.f(x) + a.f(x)
        return A(g)

    def __mul__(self, a):
        g = lambda x: self.f(x) * a.f(x)
        return A(g)

    def __call__(self, x):
        return self.f(x)

class B(A):

    def __add__(self, b):
        g = lambda x: self.f(b.f(x))
        return B(g)

class C(A):

    def __mul__(self, c):
        g = lambda x: (self.f(x), c.f(x))
        return C(g)

k1 = A(lambda x: x**2)
k2 = B(lambda x: 2*x + 1)
k3 = C(lambda x: 2**x)
k4 = A(lambda x: A(lambda y: x + y))
h = k3 * (k1 + (k2 + k4(2)))
```

Effettuando la chiamata di `h` con un valore `x` quale delle seguenti funzioni viene calcolata?

☐ $2^x(x^2 + 2x + 5)$

☐ $(2^x, x^2 + 2x + 5)$

☐ $(2^x, x^2 + 2x + 3)$

☐ $2^x(x^2 + 2x + 3)$

Domanda 8

Dato il seguente codice Python:

```
def d(a, b, c):  
    if a():  
        return b()  
    return c()  
  
def f(g, x, h, d):  
    return d(lambda: x==1 or x==0,  
             lambda: 1,  
             lambda: g(f, x-1, x-2, h, d))  
  
def g(f, x, y, h, d):  
    return h(f(g, x, h, d), f(g, y, h, d))
```

$F = \text{lambda } n, k: f(g, n, \text{lambda } x, y: x + y + k, d)$

Quale delle seguenti affermazioni descrive correttamente la funzione F ?

- | | |
|--|--|
| ☐ Computa in modo ricorsivo il numero di partizioni di dimensione k di un insieme di n elementi | ☐ Computa successioni "Fibonacci-like" in cui ogni n -esimo termine della sequenza è la somma dei due precedenti e di k |
| ☐ È la funzione identità sui valori pari e un $n+1$ per i valori dispari | ☐ Computa la funzione fattoriale in cui ogni termine è anche moltiplicato per k |
-
-
-

Domanda 9

Dato il seguente codice Python:

```
def deco(f):  
    def wrapper(g):  
        def k(*args, **kwargs):  
            def q(n):  
                if (type(n) == int or type(n) == float):  
                    return f(n)  
                return n  
            lst = list(map(q, args))  
            return g(*lst, **kwargs)  
        return k  
    return wrapper
```

```
@deco(lambda x: 0 if x < 0 else x)  
def f(a, b, c, d):  
    def p(x):  
        return a*x**3 + b*x**2 + c*x + d  
    return p
```

Quale sarà l'effetto del decoratore sulla funzione ritornata di `f`?

- | | |
|--|--|
| ☐ Azzera i coefficienti interi positivi del polinomio computato da <code>p</code> | ☐ Modifica il primo coefficiente di <code>p</code> dato che gli argomenti successivi sono passati in <code>kwargs</code> |
| ☐ Azzera i coefficienti negativi del polinomio computato da <code>p</code> | ☐ Non modifica il comportamento di <code>p</code> dato che <code>args</code> è una lista di liste e quindi il controllo fatto sul tipo fallirà sempre |
-
-
-

Domanda 10

Dato il seguente codice Python:

```
def f(g):
    n = 1
    lst = []
    while True:
        k = g(n)
        lst.append(k)
        for h in lst:
            yield(next(h))
        n += 1
```

```
def p(n):
    i = 1
    while True:
        yield n**i
        i += 1
```

`h = f(p)`

Quale affermazione è corretta rispetto a `h`?

- | | |
|---|---|
| ☐ Genera le sequenze $n^1, n^2, \dots, n^n$ al crescere di n | ☐ Genera la sequenza infinita delle potenze di 1 dato che non esce mai dalla prima iterazione del <code>while</code> |
| ☐ Genera tutti i polinomi a coefficienti interi in ordine lessicografico | ☐ Genera le sequenze $1^n, 2^{n-1}, \dots, (n-1)^2, n$ al crescere di n |
-
-
-

Parte 2

Esercizio 1

Si completi la seguente funzione C che ha i seguenti argomenti:

- `M` è un puntatore a una matrice $n \times n$ di `float`;
- `K` è un puntatore a una matrice $m \times m$ di `float` (si assuma m molto minore di n);
- `Mout` è un puntatore a una matrice $(n-m) \times (n-m)$ di `float` inizialmente tutti zero;
- `n` è un intero che indica il lato della matrice `M`;
- `m` è un intero che indica il lato della matrice `K`;

La funzione deve riempire la matrice `Mout` calcolando una convoluzione tra `M` e `K` per gli indici di `M` tra 0 e $n - m - 1$ sia per le righe che per le colonne. Ovvero:

$$Mout_{i,j} = \sum_{h=0}^{m-1} \sum_{k=0}^{m-1} M_{i+h,j+k} K_{h,k}$$

per $0 \leq i < n - m$ e $0 \leq j < n - m$. Il riempimento della matrice `Mout` deve essere parallelizzato con OpenMP in modo che il parallelismo sia adeguatamente sfruttato (i.e., non facendo svolgere le operazioni a un solo thread).

da riempire alla pagina successiva

[illegible]

Esercizio 2

Si scriva il metodo `__call__` della classe `A` che, dato un valore `x` si composta nel seguente modo:

- Se `x` e `v` sono uguali ritorna `b`;
- Altrimenti si richiama con argomento `g(x)` e combina il risultato ottenuto dalla chiamata col valore `x` tramite una chiamata alla funzione `k` che prende due argomenti e ritorna il valore ottenuto.

Questo permette, per esempio, di ottenere il calcolo del fattoriale creando una istanza di `A` come segue:

```
g = lambda x: x-1
k = lambda x, y: x * y
factorial = A(g, k, 1, 1)
print(f"Fattoriale di 10: {factorial(10)}")
```

da riempire alla pagina successiva

class A:

```
def __init__(self, g, k, v, b):  
    self.g = g  
    self.k = k  
    self.v = v  
    self.b = b
```

```
def __call__(self, x):
```
